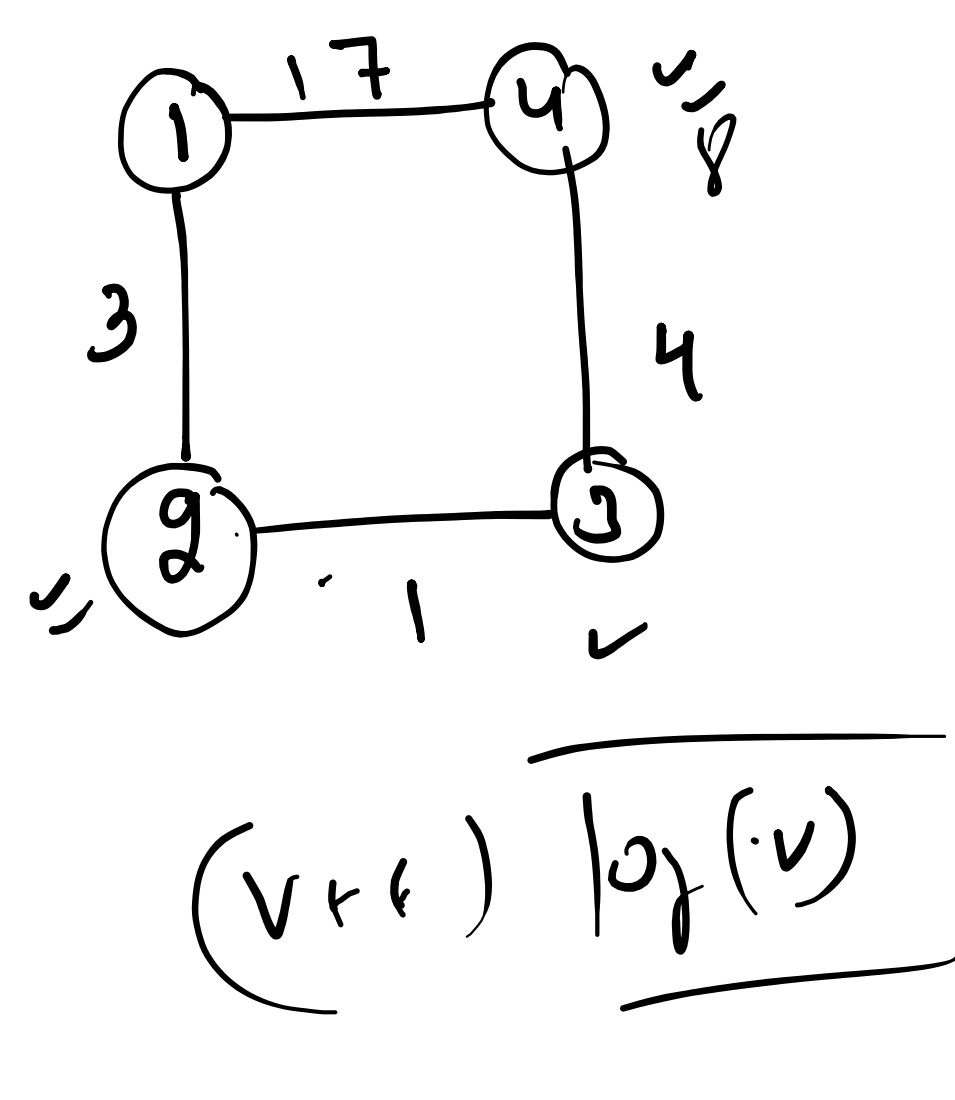
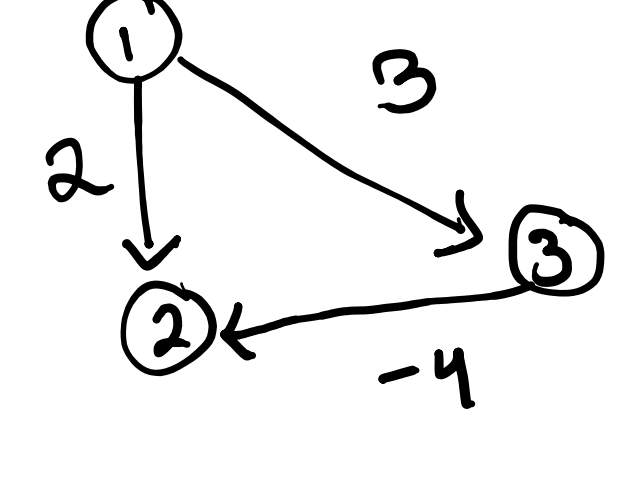




1	14	0
2	12	3
4	14	17
3	123	14
4	1234	8

123
7

1 1 @ 0
2 12 @ 3
3 12 @ 4
1 2 3 7 @ 8



src=1

v	x	path	cost
1	1	1	0
2	12	12	2
3	13	13	3

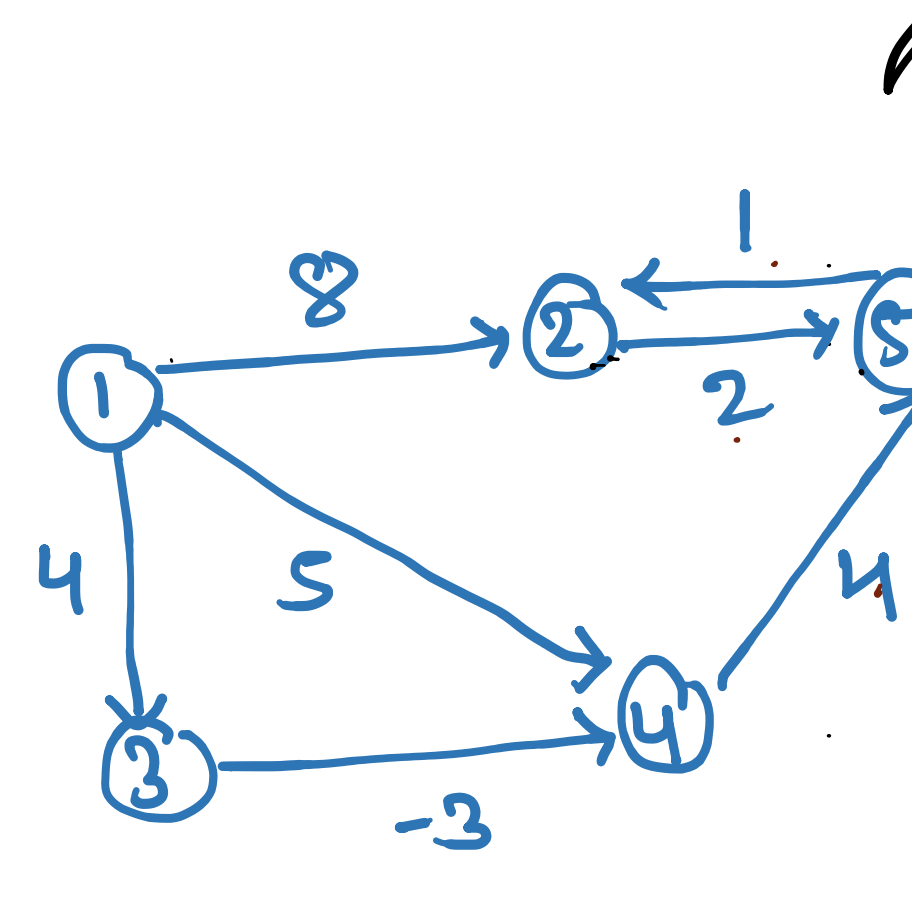
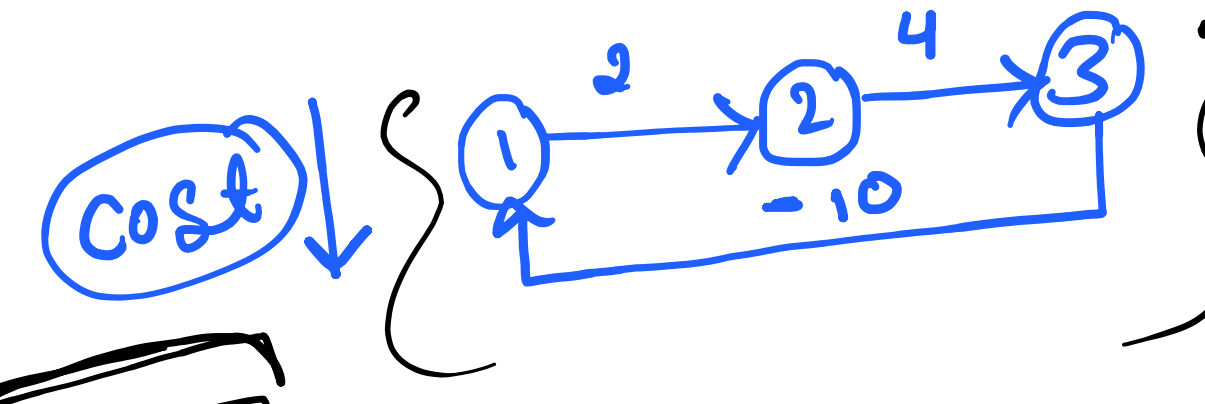
123

dist[u] undist[u]

$\begin{bmatrix} 1 & 1 & @ & 0 \\ 2 & 12 & @ & 2 \\ 3 & 13 & @ & 3 \end{bmatrix} \rightarrow 132 \rightarrow -1$

① Bellman-Ford

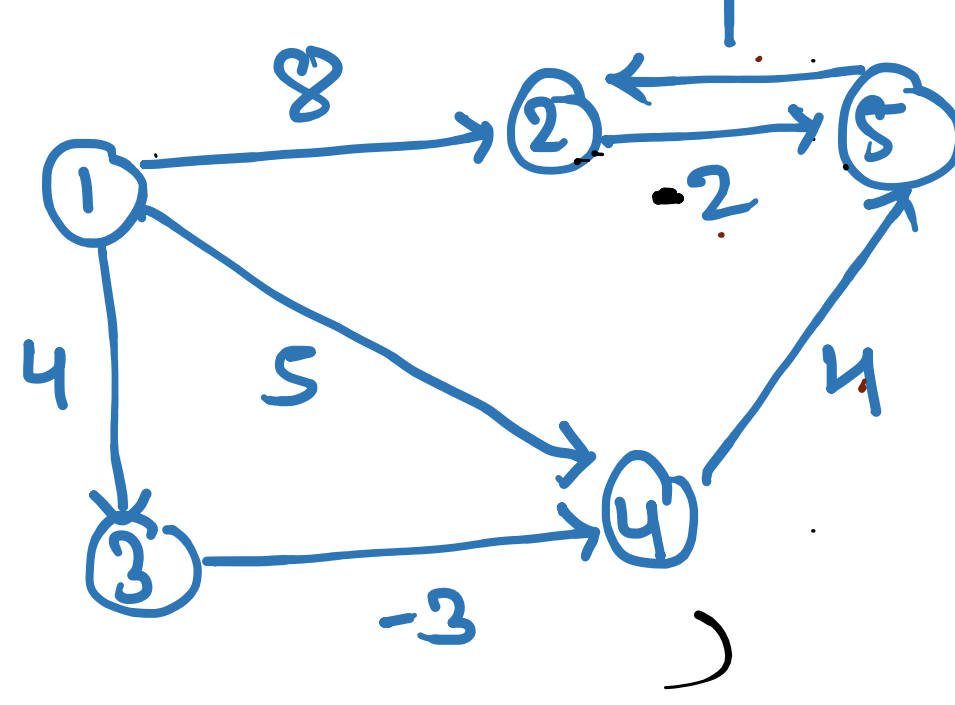
② How can it find -ve cycle



1 -> 2 8
1 -> 3 4
1 -> 4 5
3 -> 4 -3
4 -> 5 4
2 -> 5 2
5 -> 1

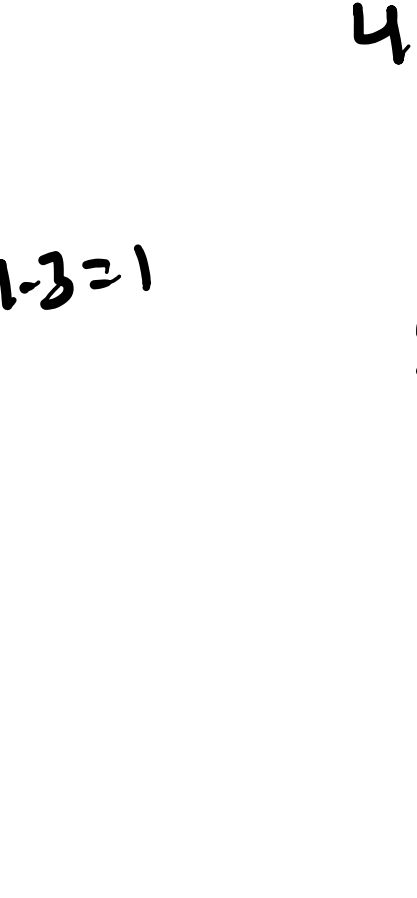
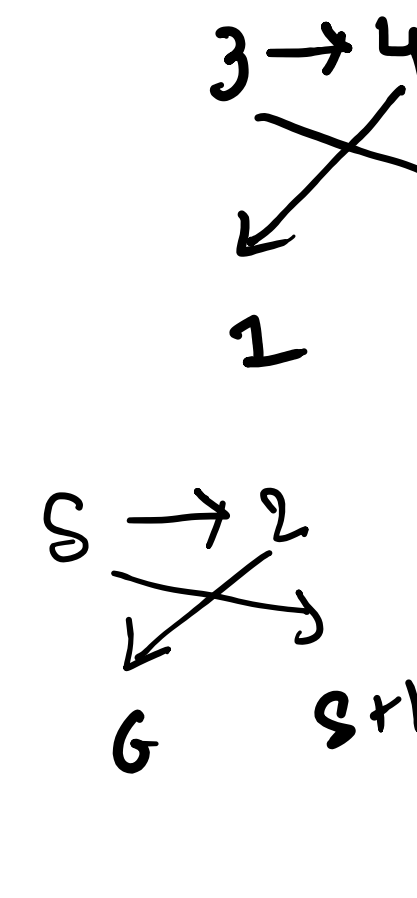
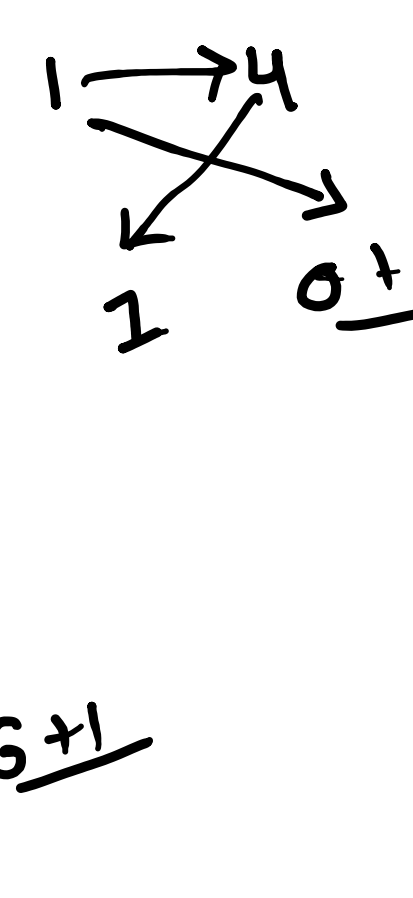
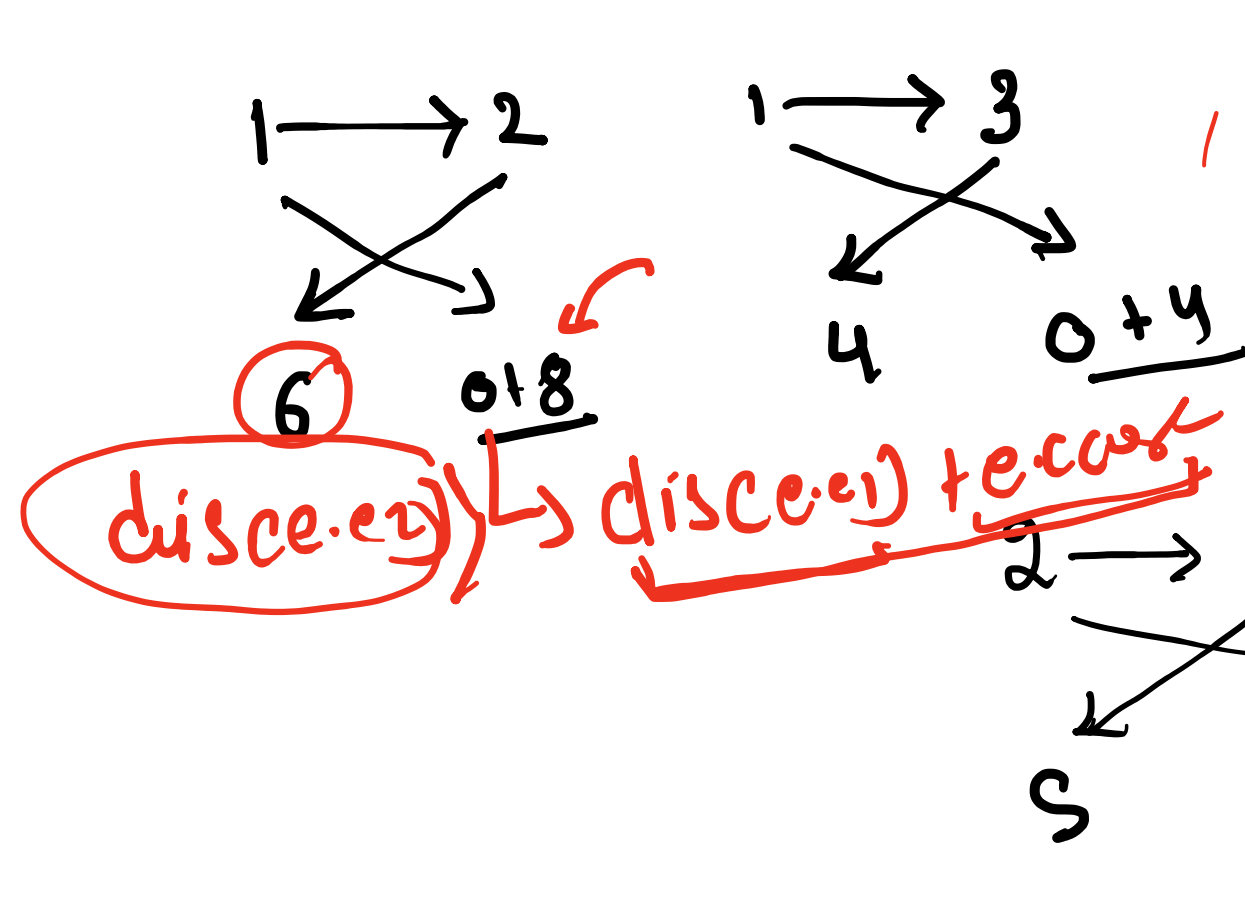
e_1, e_2 cost
[1, 2, 3, 4, 5]
for (int e1: map.keySet()) {
for (int e2: map.keySet()) {
int cost = map.get(e1).get(e2);
}}

$dis[e.e2] > dis[e.e1] + e.cost$

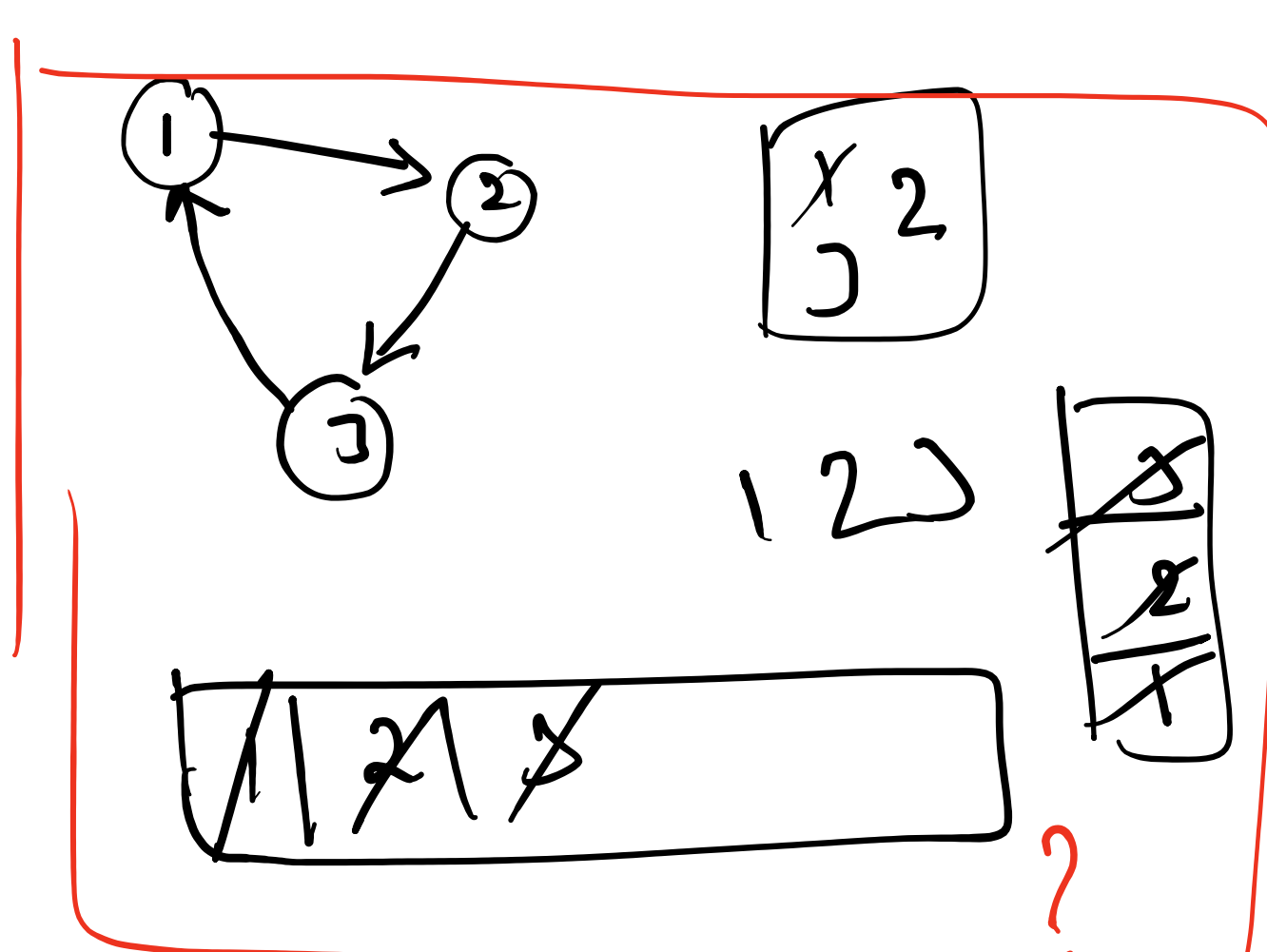
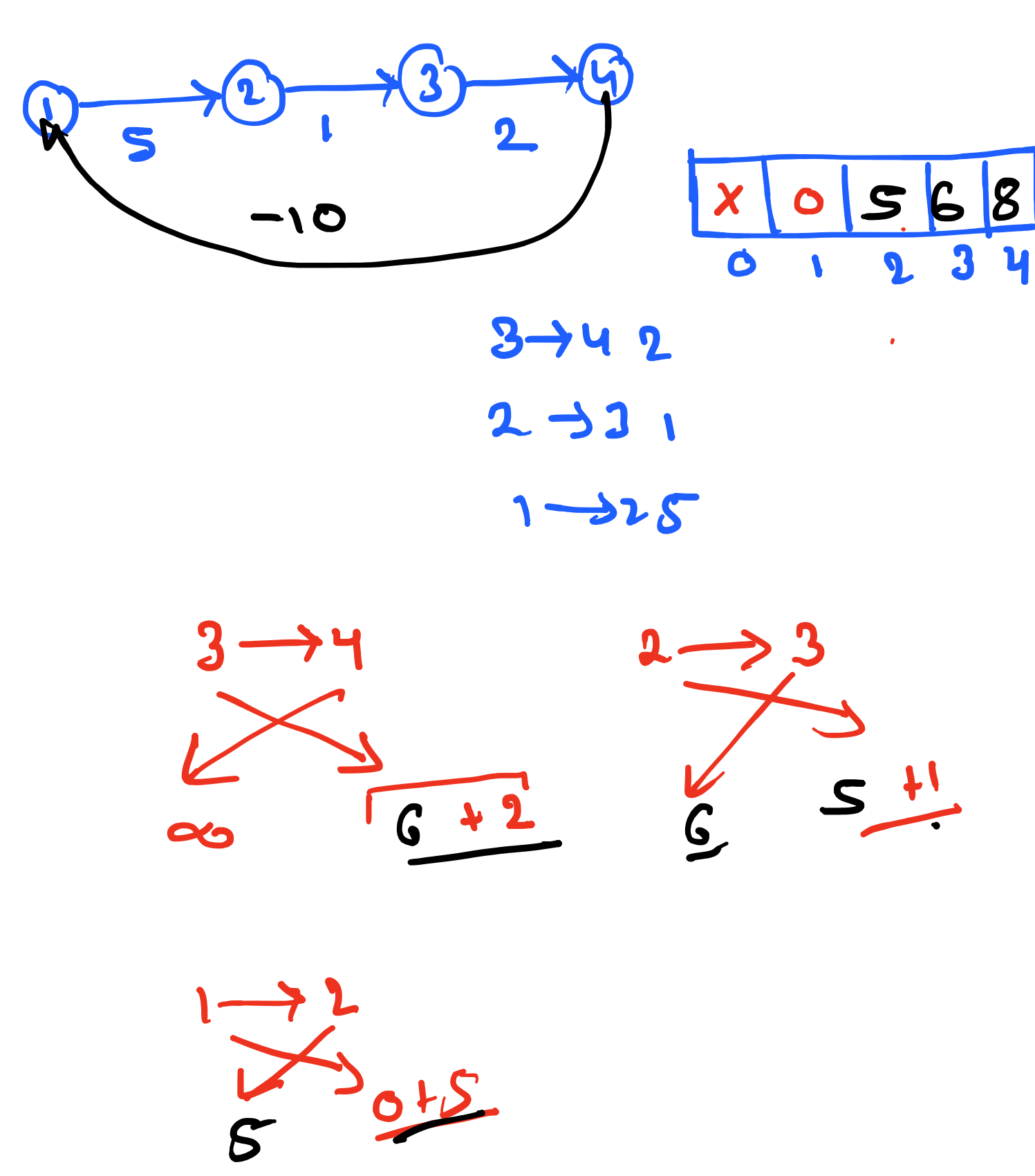


1 -> 2 8
1 -> 3 4
1 -> 4 5
3 -> 4 -3
4 -> 5 4
2 -> 5 2
5 -> 1

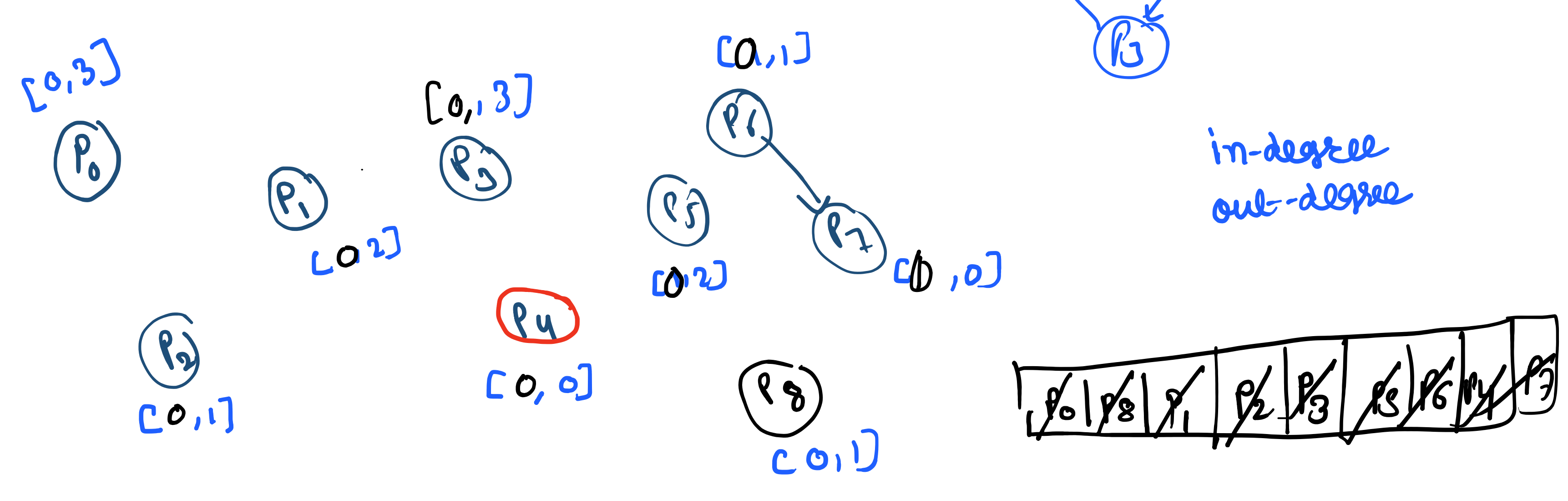
X	0	6	4	1	5
0	1	2	3	4	5



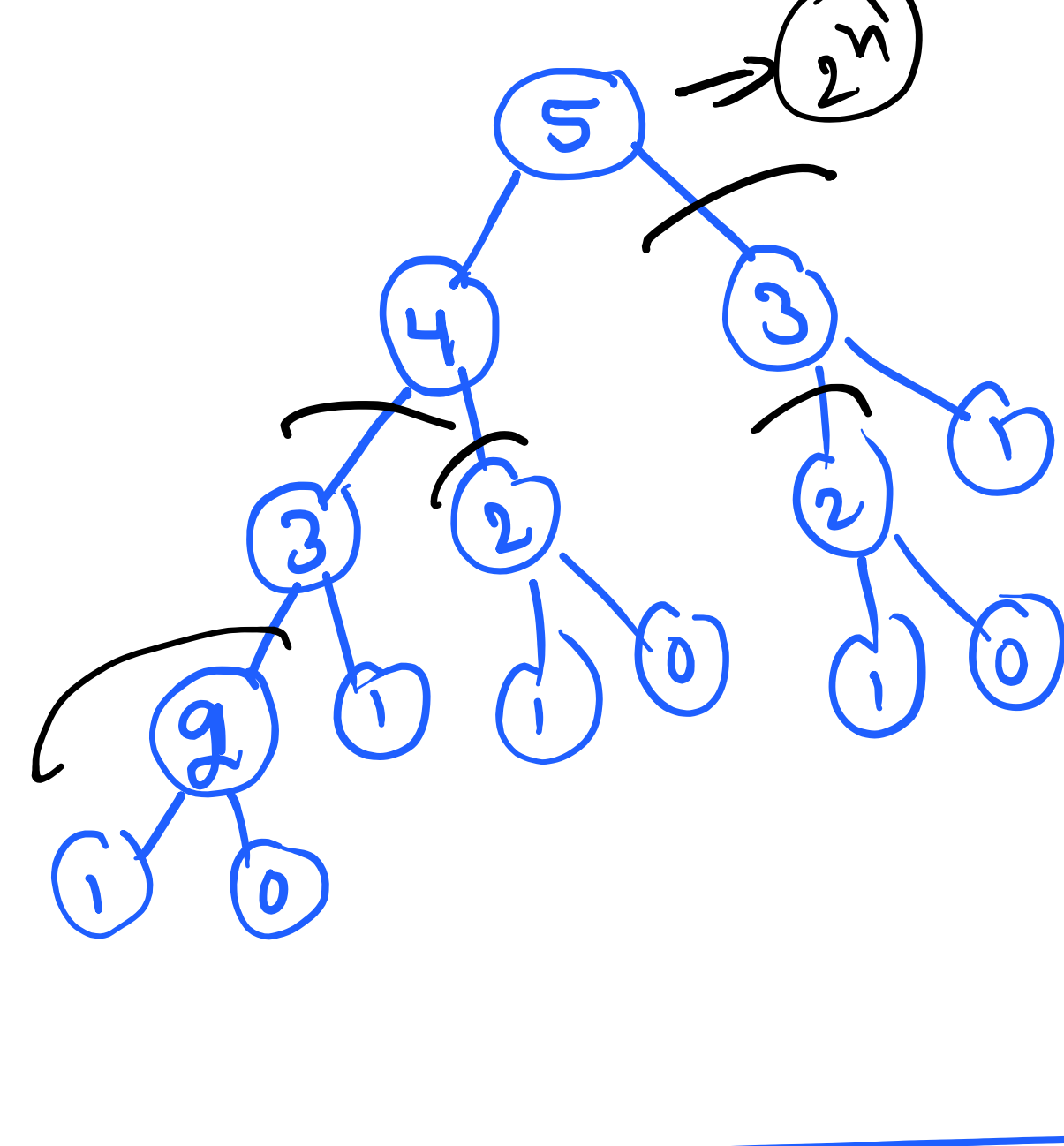
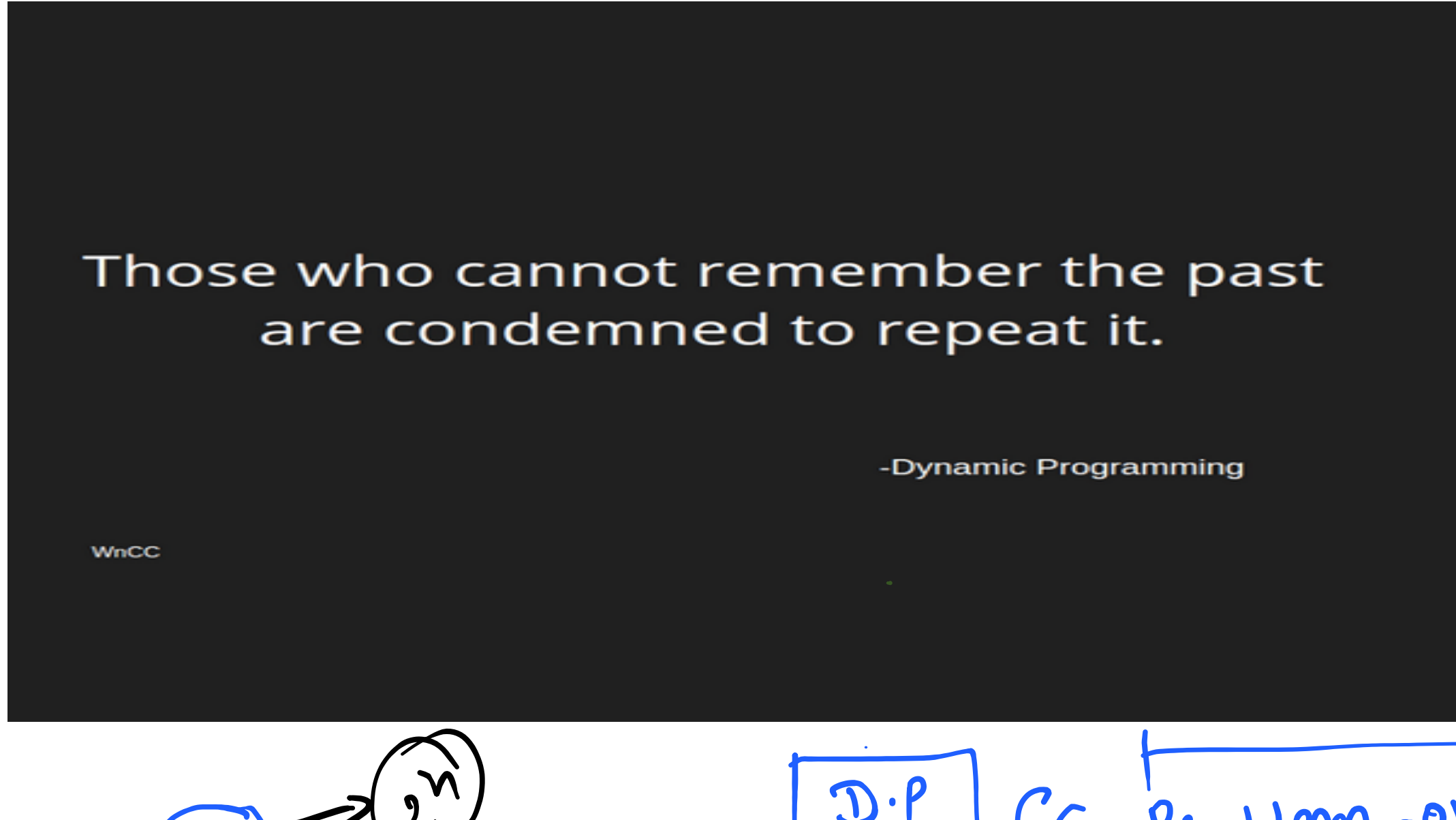
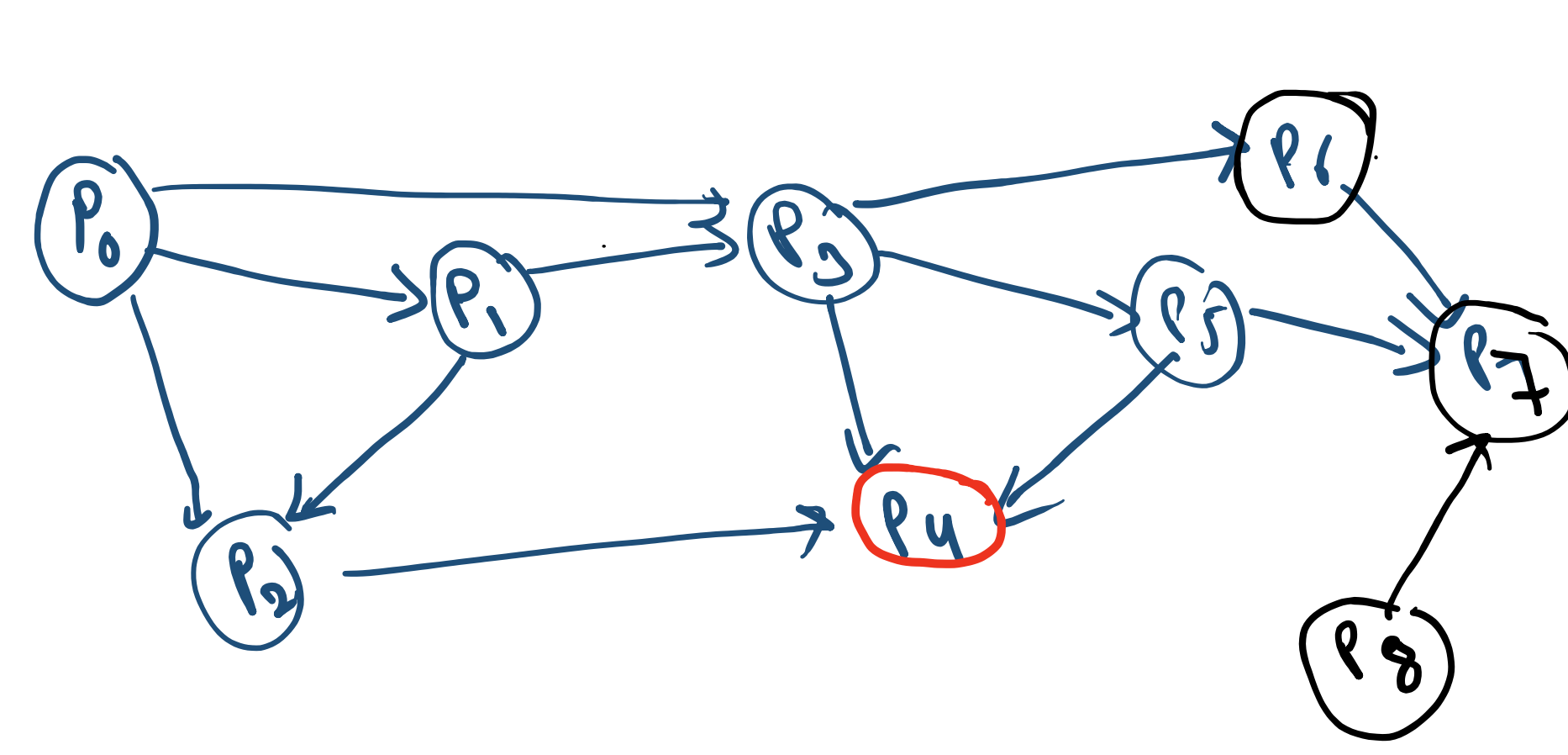
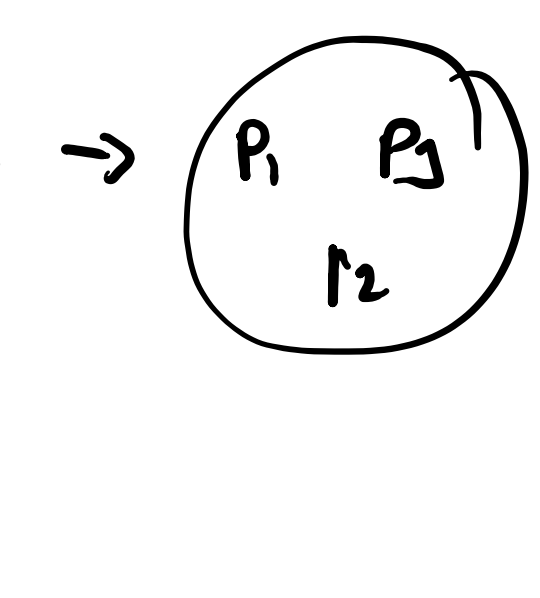
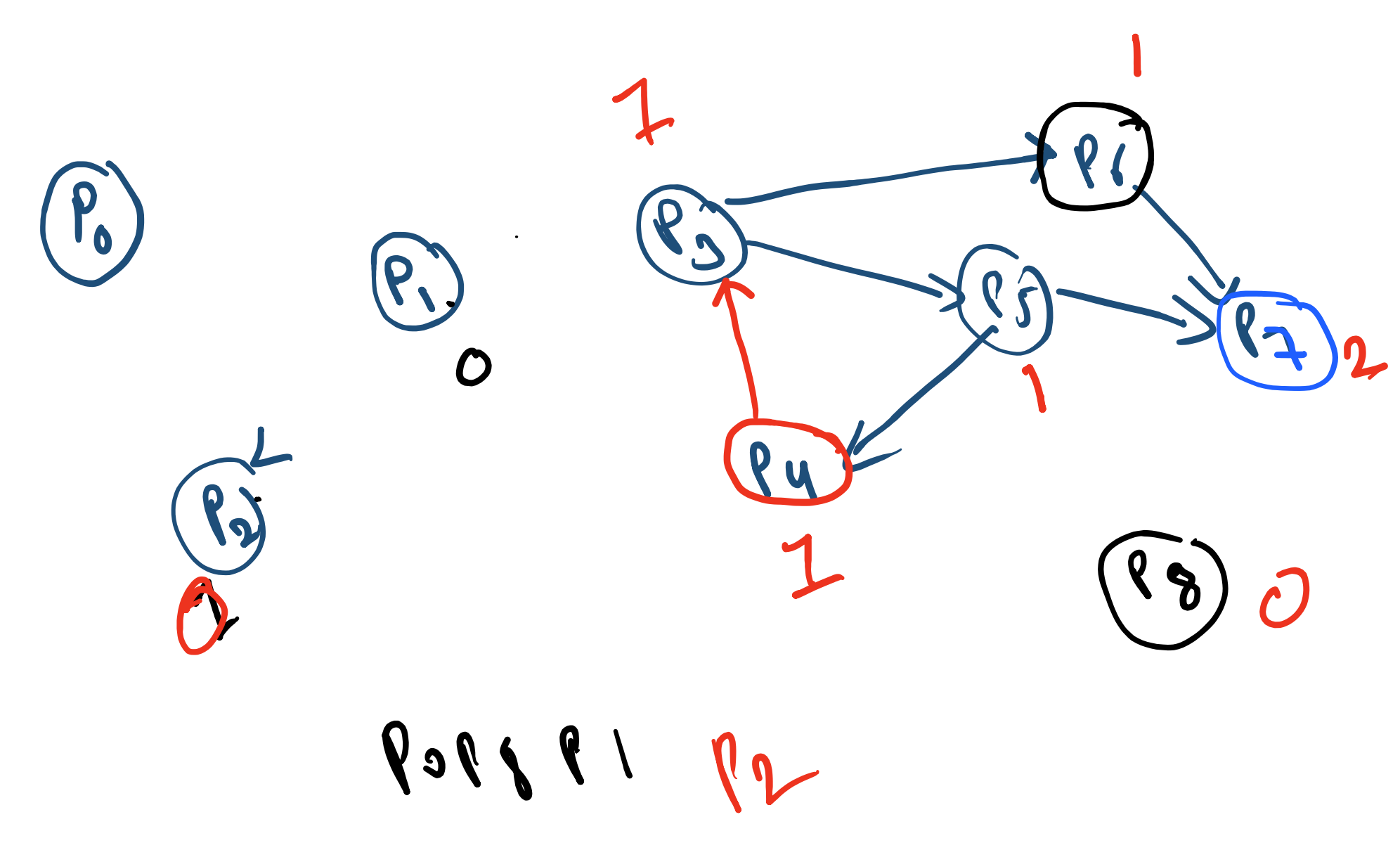
```
public void BellManFord(int src) {
    int v = map.size();
    int[] dis = new int[v + 1];
    for (int i = 1; i < v; i++) {
        if (i != src) {
            dis[i] = 999999;
        }
    }
    List<EdgePair> ll = getAllEdge();
    for (int i = 1; i < v; i++) {
        for (EdgePair e : ll) {
            if (dis[e.e2] > dis[e.e1] + e.cost) {
                dis[e.e2] = dis[e.e1] + e.cost;
            }
        }
    }
    for (int i = 1; i < v; i++) {
        System.out.println(i + " " + dis[i]);
    }
}
```



Topological Sort Kahn Algo
in-degree out-degree



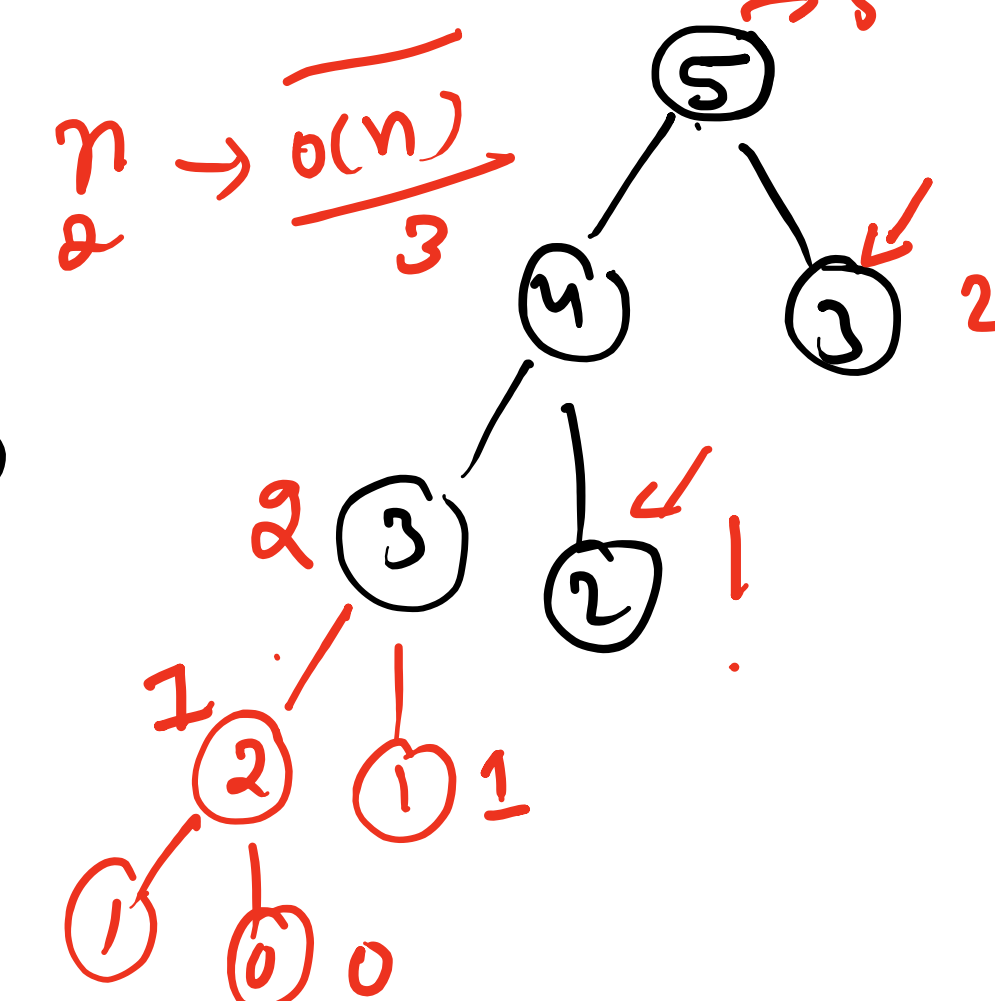
P0 P8 P1 P2 P3 P5 P6 P4 P7



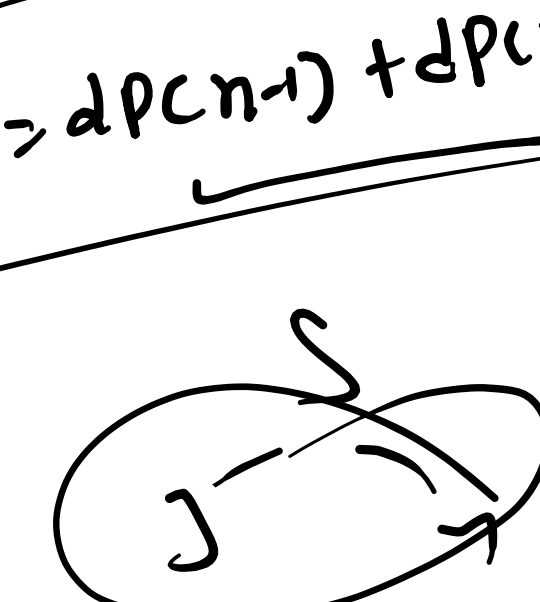
① Problem-overlapping
② optimal sub str

① Top-down -> Recursive -> memoization
② Bottom-up -> iterative tabulation

```
public static int fib(int n) {
    if (n == 0 || n == 1) {
        return n;
    }
    int f1 = fib(n - 1);
    int f2 = fib(n - 2);
    return f1 + f2;
}
```



$i=2$ $dpc[2] = dpc[1] + dpc[0]$
 $i=3$ $dpc[3] = dpc[2] + dpc[1]$
 $i=4$ $dpc[4] = dpc[3] + dpc[2]$
 $dpc[5] = dpc[4] + dpc[3]$



0	1	1	2	3	5
0	1	2	3	4	5

